

Mode Collapse in RL-Trained Theorem Provers

Structural Guidance as a Diagnostic and Remedy

Zachary Burton

arXiv: 2601.16172

MIT → MPhil in Advanced Computer Science, Cambridge

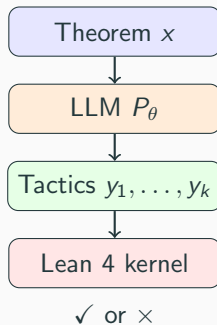
Background: Neural Theorem Proving

Setup:

- Formal theorem x in Lean 4 (dependent type theory)
- LLM P_θ generates tactic proof y
- Lean kernel verifies: $\text{check}(x, y) \in \{\top, \perp\}$
- Sample k attempts, accept if any verify

Model: DeepSeek-Prover-V1.5-RL

- 7B params, GRPO-trained (SFT $\rightarrow$ RL)
- Evaluated on miniF2F: 244 competition-level theorems
- Pinned to Lean v4.9.0-rc1 (training env)



Hypothesis: Mathematical Intuition Should Help

The core idea:

A human mathematician knows that some problems want induction, others want contradiction, others want algebraic simplification.

RL-trained LLMs lack this intuition. They converge on a narrow default strategy and repeat it—even when it fails.

If we inject a mathematician's sense of “how to attack this,” the model should do better.

Evidence—sampling saturates:

Budget	Solved	Δ
$k = 16$	38 / 244	—
$k = 32$	42 / 244	+4
$k = 64$	42 / 244	+0

Doubling the budget: **zero new proofs.**

The model isn't exploring—it's stuck.

More compute alone won't fix this.

Mathematical guidance might.

Method: Expert Hints as Tactic Prefixes

Idea: Encode a mathematician's intuition as a tactic prefix—tell the model *how to start* the proof.

$P_\theta(y \mid x, s)$: model completes proof of theorem x given hint s .

15 hints reflecting common strategies:

0. (empty — no hint)
1. `simp`
2. `intro`
3. `intros`
4. `constructor`
5. `refine ?_`
6. `norm_num`
7. `linarith`
8. `nlinarith`

The analogy:

A student comes to you stuck on a proof. You don't solve it for them—you say “have you tried induction?” or “what if you simplify first?”

That small nudge is often enough. The model can *execute* tactics well; it just picks the wrong *strategy*.

Properties:

Concrete Example: Hints Unlock New Proofs

Prove $12 \mid 4^{n+1} + 20$. Without hint: model tries `norm_num` variants, fails all 16 attempts.

Hint `simp` $\rightarrow$ model completes:

```
theorem induction_12dvd4expnp1p20
  (n : N) : 12 ∣ 4^(n+1) + 20 := by
  simp
  induction n with
  | zero => simp
  | succ n ih =>
    simp_all [Nat.mul_mod,
              Nat.pow_succ]
    omega
```

The hint nudged the model toward `simp` + `induction`—a strategy it never tried on its own.

Another: $ab=180$,

$$2(a+b)=54 \Rightarrow a^2+b^2=369$$

Without hint: tries `field_simp`, loops, fails.

Hint `ring` $\rightarrow$ model completes:

```
theorem mathd_algebra_141
  (a b : R)
  (h1 : a * b = 180)
  (h2 : 2 * (a + b) = 54) :
  a^2 + b^2 = 369 := by
  ring
  nlinarith
```

13 theorems solved **only** via hints (never by i.i.d. sampling).

Results + Ablation

Main results:

	@16	@32	@64
i.i.d. baseline	38	42	42
skeleton hints	55	58	60
Relative gain	+45%	+38%	+43%

Baseline **flatlines** (42 $\rightarrow$ 42).

Hints **keep climbing** (55 $\rightarrow$ 58 $\rightarrow$ 60).

Ablation (Pass@16): is *any* perturbation enough?

Condition	Solved	vs. base
C2: random tokens	25	−34%
A: i.i.d. (control)	38	—
C1: paraphrases	38	±0%
B: tactic hints	55	+45%

Collapse is in *tactic selection*, not instruction following. Only structural hints help.

Base vs. RL: RL Creates Capability and Collapses It

Same architecture, same
pre-training—with and without GRPO.

	@16	@32	@64	Hint effect
BASE i.i.d.	0	0	0	None
BASE hints	0	0	0	
RL i.i.d.	38	42	42	+43%
RL hints	55	58	60	

Base model outputs sorry for every
theorem—**zero proof capability**.

The narrative:

1. RL *creates* proof capability
(BASE $\rightarrow$ RL: 0 $\rightarrow$ 38+)
2. RL simultaneously *narrows* the
strategy space (mode collapse)
3. Structural hints recover diversity
that RL eliminates

Broader point: RL gives the model *skill*
but not *judgment*. Mathematical experts
can supply the judgment that RL strips
away.

Research proposal: studying **proof search itself**—how proof states evolve, how tactic sequences branch via subgoal decomposition, and how RL and verifier feedback shape exploration.

Open questions:

- What forms of expert guidance help most? (tactic-level, lemma-level, strategic?)
- Can we *learn* to select hints from proof structure? (hint retrieval)
- Can RL preserve mathematical judgment, not just maximize reward?

Planned modules:

- L81 (Proof Assistants)
- L313 (HoTT & Univalent Foundations)
- L330 (Quantum Complexity Theory)
- R252 (Deep Learning Theory)

Paper status: Targeting AI4Math @ ICML 2026

arXiv:2601.16172